

Assignment 2 - Data Structures

Nursultan Malgazhdar, SE-2516 | 2026-10-05

This project compares three ways to store integers. `DynamicArray` keeps values in an array. `MyLinkedList` keeps values in nodes connected by links. `MinHeap` keeps the smallest value at its root. All values are primitive `int` values, and the structures do not use ready-made Java collections.

The array and heap start with 16 cells and double their capacity when full. They do not shrink after removal. The list is singly linked and has both a head and a tail. Invalid indices and empty-heap access throw the required exceptions.

Time complexity

Operation	Best	Average*	Worst	Extra space	Reason
Array <code>add(x)</code>	$\Theta(1)$	$\Omega(1), O(n)$	$\Theta(n)$	$\Theta(n)$	Usually one store; a full array copies n values.
Array <code>add(i,x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Move values after i ; grow if full.
Array <code>remove(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Move values after i one position left.
Array <code>get(i)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Read the cell at index i directly.
Array <code>contains(x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Check values until a match or the end.
List <code>add(x)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	The tail points directly to the last node.
List <code>add(i,x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head/tail are direct; other indices need a walk.
List <code>remove(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Except at the head, find the previous node.
List <code>get(i)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Follow i links from the head.
List <code>contains(x)</code>	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Check nodes until a match or the end.
Heap <code>insert(x)</code>	$\Theta(1)$	$\Omega(1), O(n)$	$\Theta(n)$	$\Theta(n)$	Move up at most $\log n$ levels; growth copies n .
Heap <code>peekMin()</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	The smallest value is at index 0.
Heap <code>extractMin()</code>	$\Theta(1)$	$\Omega(1), O(\log n)$	$\Theta(\log n)$	$\Theta(1)$	Replace the root and move down if needed.

Here n is the current number of elements. Θ gives a tight growth rate, O an upper bound, and Ω a lower bound. For indexed operations, the average assumes each valid index is equally likely. For search, half the queries are misses.

*Append and heap insertion also depend on whether the array is full. Their average columns show safe bounds, not one exact random-input average. Over a long sequence, array append is $\Theta(1)$ amortized and heap insertion is $O(\log n)$ amortized. Amortized cost spreads occasional expensive resizing across many operations.

Extra space means memory used during one operation, beyond the stored data. Growing an array needs a new array. Stored array memory is $\Theta(\text{capacity})$, which can exceed the current size after removals; the list stores $\Theta(n)$ nodes. The bonus `buildHeap` uses $\Theta(n)$ time and $\Theta(n)$ space for its copy.

Correctness and benchmark setup

1. `DynamicArray.contains(value)`

Invariant: before iteration i , all elements before i have been checked and none equals the target. The array and its size do not change.

Initialization: i is 0, so the checked part is empty and the statement is true.

Maintenance: if the current value matches, returning true is correct. Otherwise, i increases and the checked part grows by one.

Termination: i increases each time, so the loop either finds a match or reaches size. At size, every element has been checked.

Conclusion: the method returns true exactly when the target exists, and false otherwise.

2. `MinHeap.bubbleDown(index)`

Invariant: in the subtree handled by this call, only the current node may be larger than its children. Both child subtrees are heaps. The nodes already fixed above it are no larger than the values below them. No values are added or removed.

Initialization: after extraction, only the new root may be misplaced. In Floyd's build, the child subtrees have already been fixed.

Maintenance: the method chooses the smaller child. If the current value is no larger, this part is already a heap. Otherwise, a swap fixes the current position and moves the possible problem down one level.

Termination: each swap moves downward, so the method eventually reaches a leaf or a correct position.

Conclusion: the subtree becomes a min-heap with the same values. After extraction, this restores the remaining heap.

How the measurements were made

The sizes are 100, 1,000, 10,000 and 100,000. Each run uses `new Random(42)`, so both sequences receive the same data and queries. The JVM first runs 20 practice cycles at $n = 1,000$. Each case then has two more warm-ups and five timed runs. The reported time is the middle of the five sorted times, called the median. There are 36 cases and 180 saved measurements.

For W1-W3, filling the structure and making the queries happen before timing, and counters reset after filling. W4 includes both insertion and extraction, plus a check that the extracted values never decrease. The code uses the returned values so the JVM cannot simply discard the measured work.

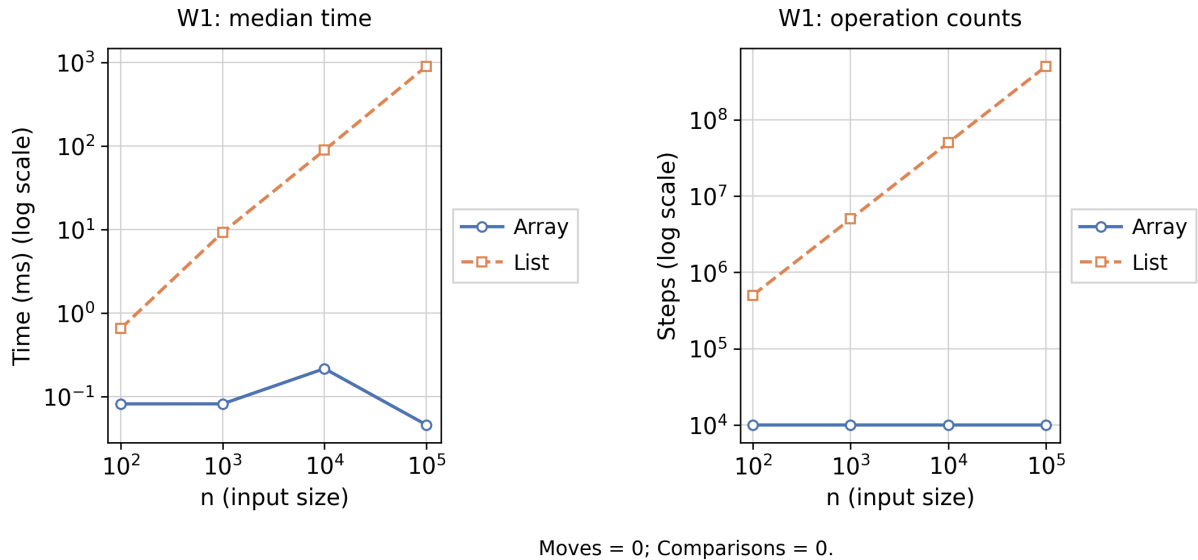
A step is one array-cell read or one next-link read in the list. A move is one relocation of an existing array value, or one change to head, tail or next in the list. A comparison compares two element values. The first store of a new array value, loop bounds and index checks are not counted as moves or element comparisons. These counters are updated inside the methods.

Tests: all 47 JUnit 5 tests passed. Machine: AMD Ryzen 5 7535HS with Radeon Graphics; Windows 11; Java 21.0.12.1. Raw runs and VM details are saved in results/.

Results: access and search

W1. Random access

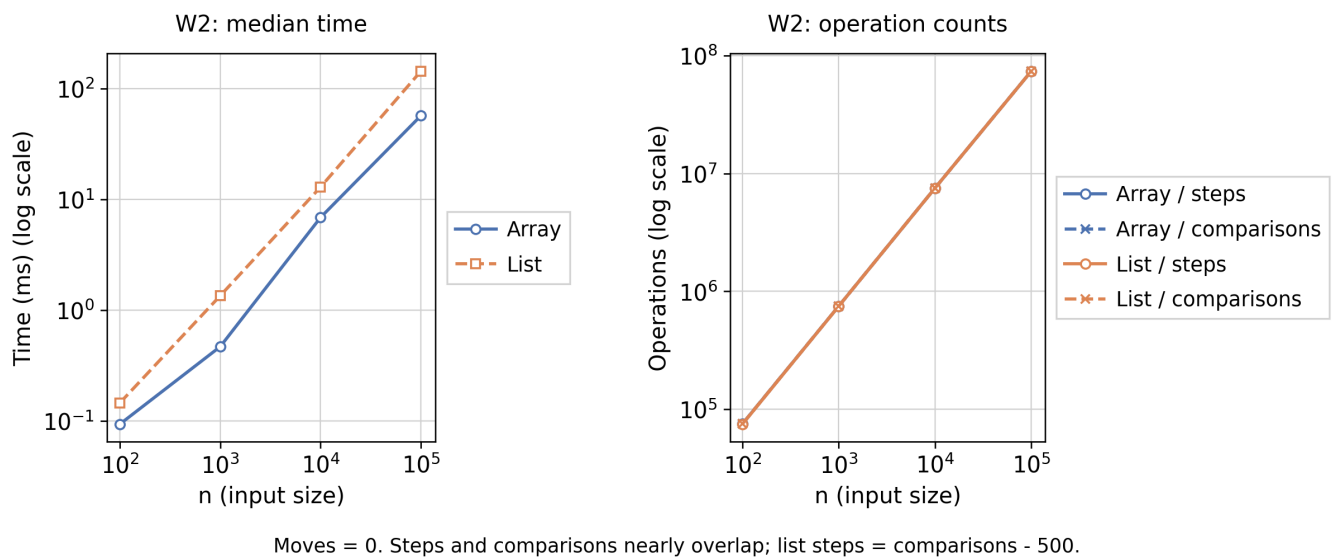
After filling each structure, the benchmark makes 10,000 `get(index)` calls at random indices.



At $n = 100,000$, the array takes 0.046 ms and the list takes 898.768 ms. The array reads 10,000 cells; the list follows 504,930,938 links. The array goes directly to an index, while the list starts from the head.

W2. Search

There are 1,000 `contains(value)` queries: 500 values are present and 500 are guaranteed absent.

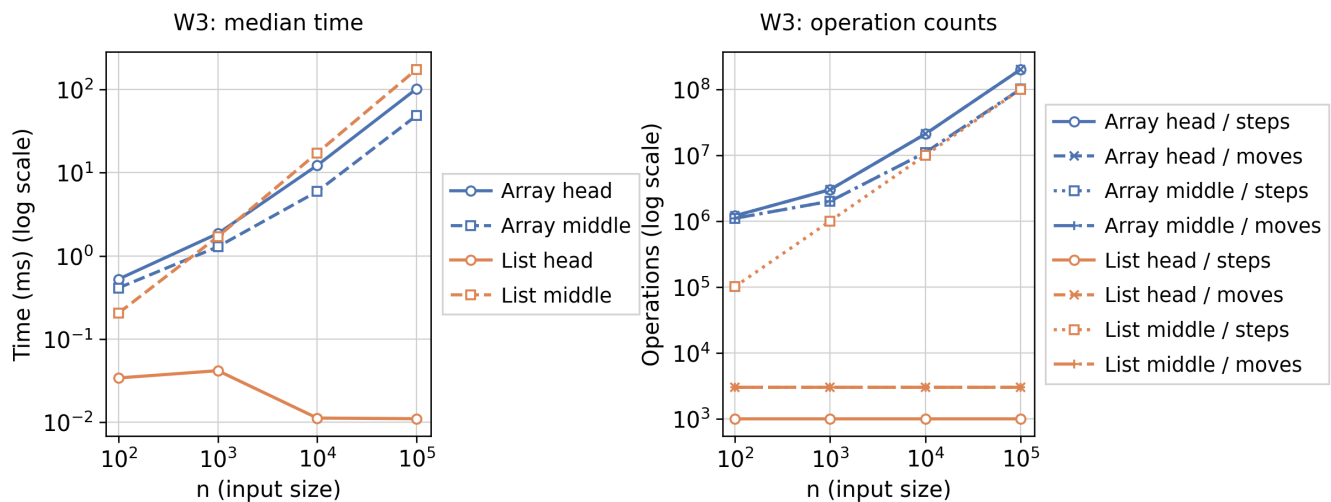


At $n = 100,000$, the times are 56.849 ms for the array and 143.468 ms for the list. Both make 73,835,543 comparisons. The list has 500 fewer steps because a successful search stops before reading the next link. Almost equal count lines overlap in the graph.

Results: updates and priority processing

W3. Insert and remove

Each variant makes 1,000 insertions, then 1,000 removals. Head uses index 0; middle uses the original $n / 2$ throughout.

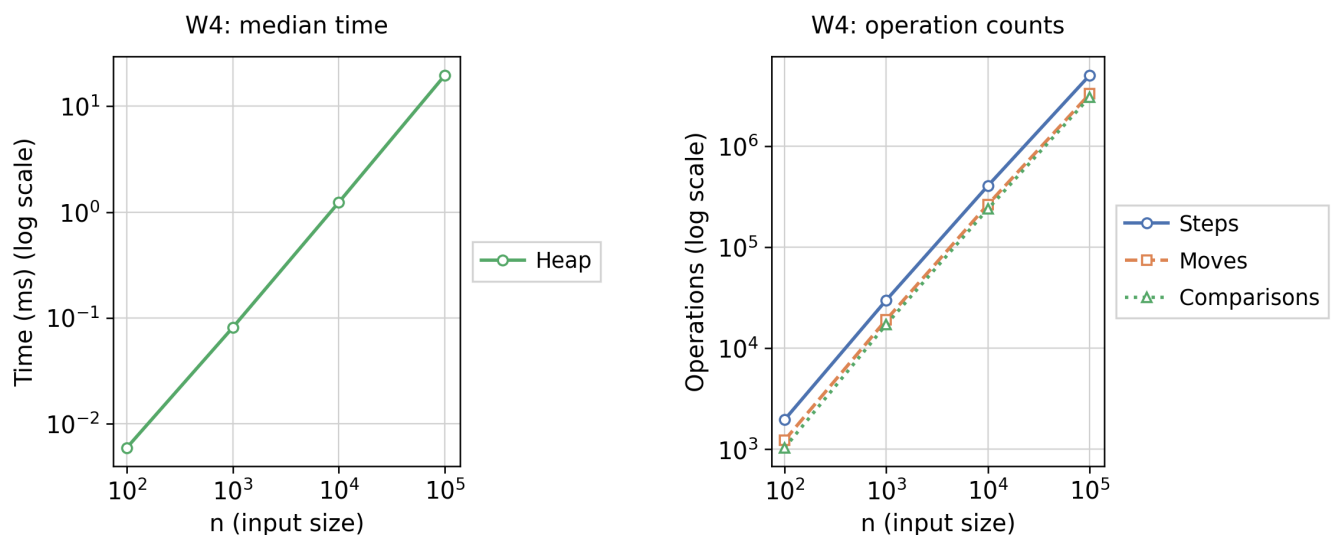


Comparisons = 0. Array steps = moves + 1,000; its curves nearly overlap.

At $n = 100,000$, head updates take 102.196 ms for the array and 0.011 ms for the list. Middle updates take 48.633 and 174.951 ms. The list changes 3,000 links in each variant, but finding the middle adds a long walk. Array steps and moves nearly overlap because they differ by only 1,000.

W4. Priority processing

The heap receives n values, then `extractMin()` is called n times. The output is checked to be in non-decreasing order.



At $n = 100,000$, the total time is 19.660 ms and the heap makes 3,059,125 element comparisons. The heap is empty at the end. Growth copies are included in its counters; the benchmark's ordering checks are not.

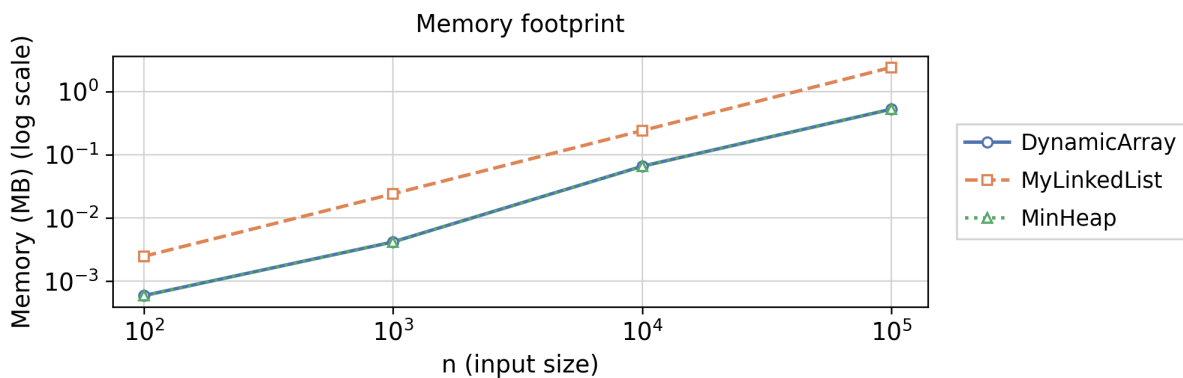
Discussion and bonus tasks

DynamicArray is a good choice for reading values by index because it reaches the requested cell directly. MyLinkedList must follow links from the head, so a large index takes more work. Array values are next to each other in memory, and a CPU cache line can bring several nearby values into the processor at once. This also helps a simple scan, even when both structures check the same number of values.

List nodes have extra headers and links, and the processor must read each link before it knows where to go next. More objects also give the garbage collector more objects to manage. The timings fit these explanations, but this benchmark does not measure cache misses or garbage-collection pauses separately. A list is useful for repeated changes at the head and, with a tail pointer, for appending values.

Changing the middle by index is slower because the list must first find that position. MinHeap is useful when the next job should always have the smallest priority value. Resizing sometimes makes an array or heap insertion expensive, but doubling spreads that copying cost over many insertions. Warm-ups and median times reduce noise, but background activity and JVM optimisation can still change small timings.

Bonus A. Memory use



JOL reports 0.524 MB for the array, 2.400 MB for the list and 0.524 MB for the heap at $n = 100,000$. This includes the structure, counters and spare cells. A node uses 24 bytes here: 12 for its header, 4 for the int, 4 for the link and 4 for alignment. The arrays keep 131,072 cells. MB means 1,000,000 bytes. JOL could not attach its Serviceability Agent, so no object-address claims are made.

Bonus B. Building a heap

At $n = 100,000$, Floyd's buildHeap takes 2.308 ms and 188,424 comparisons. Repeated insertion takes 3.098 ms and 227,662 comparisons. Floyd starts near the bottom, where most nodes need little work, and then works upwards. The total is $\Theta(n)$, including the input copy. Repeated insertion can take $O(n \log n)$ in the worst case, but random values often move up only a short distance. Its extra graphs are in results/plots/.

Sources: the Assignment 2 course brief and OpenJDK JOL. All graph values come from the saved CSV files.